



MSA UNIVERSITY
جامعة أكتوبر للعلوم الحديثة والآداب



October University for Modern Science and Arts
Faculty of Computer Science
**Machine Learning Project Report — Phase
2**
**Braille Character Recognition using Deep Learning
and Object Detection**

Course Name: Machine Learning

Course Code: CS363

Instructor: Assoc Prof. Nermin Abd El-Wahab, Dr. Ebtsam El-Hussany

Teaching Assistant: Eng. Omnia Fawzy, Eng. Youssef Araby, Eng. Mazen Ashraf,
Eng. Ahmed Yasser

Team Members:

David Nashaat – 247687

Youssef Adel – 247121

Abanoub Elwy – 243505

Mario Nader – 248655

Submission Date: May 16, 2026

Contents

1	Introduction	2
2	Literature Review	3
3	Dataset Description	4
4	Data Preprocessing and Feature Engineering	9
5	Implemented Models	10
6	Evaluation Strategy	12
7	Results and Comparative Analysis	13
8	Error Analysis and Critical Reflection	24
9	Conclusion and Future Work	25
10	Teamwork and Task Delegation	27
11	Appendix	27

1 Introduction

Braille is a tactile writing system used by over 253 million visually impaired individuals worldwide. Each Braille character is formed by a unique combination of raised dots arranged in a 2×3 cell grid, encoding up to 63 distinct non-empty patterns representing letters, digits, punctuation marks, and common English contractions. The automated recognition of Braille characters from scanned document images is a problem of significant humanitarian importance: it enables the digitisation of Braille books, assists educators and transcribers, and can power real-time translation applications for the visually impaired community.

Braille OCR is a non-trivial fine-grained detection challenge. Characters are small relative to a full-page image (often only $\sim 24 \times 24$ pixels when a 300 DPI page is scaled to a 1024-pixel network input), the number of classes is large (63), and the dataset is heavily imbalanced because common characters such as *the*, *and*, and *for* appear far more frequently than rare contractions and punctuation marks.

Project Objective. This project implements and compares multiple deep learning and classical machine learning models for end-to-end Braille character recognition. Phase 1 focused on classical feature engineering and traditional ML classifiers applied to isolated, pre-cropped Braille cells. Phase 2 extends the pipeline with modern object detection architectures (YOLOv8s, YOLOv10s, YOLO11n/s/m) and a custom CNN, enabling both classification of isolated Braille cells and full-page character detection directly from scanned pages.

Datasets Used. Phase 1 used the Kaggle Braille Character Dataset (1,560 isolated, pre-cropped 28×28 images of 26 letter classes). Phase 2 used the Angelina Braille Interpreter Dataset [1], which provides full-page scans of Braille books with pixel-level bounding-box annotations across all 63 classes.

Models Implemented.

- **Phase 1 (Classical ML):** SVM (RBF kernel), K-Nearest Neighbours, Random Forest, Logistic Regression, and Gaussian Naïve Bayes — all trained on HOG features with PCA dimensionality reduction.
- **Phase 2 (Deep Learning):** Custom CNN (David Nashaat), YOLOv8s (Youssef Adel), YOLOv10s Grandmaster (Abanoub Elwy), and the YOLO11n/s/m family (Mario Nader).

Summary of Results. In Phase 1, SVM (RBF) achieved the best weighted F1-score of **0.8028** and ROC-AUC of **0.9877**. In Phase 2, the Custom CNN achieved **99.72%**

test accuracy on isolated 32×32 Braille crops. Among the full-page detection models: YOLOv10s (Grandmaster) achieved mAP@0.50 of **98.12%** (P: 97.16%, R: 97.87%); YOLOv8s with synthetic augmentation peaked at mAP@0.50 of **73.1%**; and YOLO11m achieved **77.62%** mAP@0.50 with 41/63 classes above 50% AP.

2 Literature Review

Paper 1 — Gabor Wavelet and Multiclass SVM for Braille Image Classification

Agustina et al. [2] address Braille letter image classification to assist visually impaired individuals. The authors apply Gabor Wavelet for feature extraction combined with a Multiclass SVM for classification. Testing on 910 Braille images, they achieve a recognition accuracy of **98.02%**. The dataset used originates from a specific Braille card format, which is a key limitation — the controlled acquisition conditions (uniform lighting, fixed distance, standard card stock) do not represent the diversity of real-world Braille documents such as embossed book pages scanned at varying DPI settings.

Relation to this project: This paper directly mirrors our Phase 1 pipeline of image-based feature extraction followed by classical ML classification on Braille data. The 98.02% accuracy sets a benchmark for our objectives and validates SVM as a highly effective choice for this problem. The feature extraction paradigm — transform raw pixels into a structured descriptor, then apply a kernel SVM — is precisely what our HOG+SVM pipeline implements, providing a principled theoretical foundation for our model selection.

Paper 2 — Deep Learning vs. Classical ML on Braille Character Recognition

Sana et al. [3] compare a deep learning approach (GoogLeNet) against classical ML classifiers including SVM, KNN, Naïve Bayes, and Decision Tree on Braille A–Z classification. SVM achieves 83% total accuracy, while KNN and Naïve Bayes reach approximately 97% with lower per-class sensitivity. The main limitation is that the dataset originates from a specific touchscreen application, restricting diversity in dot rendering and scanning artefacts.

Relation to this project: This paper directly benchmarks the same classical ML models we propose for Phase 1. Their results provide expected performance ranges and justify our model selection — SVM and KNN are both validated as competitive choices. Furthermore, the comparison of classical ML against deep learning (GoogLeNet) motivates the transition to Phase 2: deep networks offer end-to-end feature learning that can scale to

full-page, noisy scans where hand-crafted descriptors like HOG lose discriminative power.

Paper 3 — RICA and PCA-Based Feature Extraction for English Braille Patterns

Shokat et al. [4] present characterisation of English Braille patterns using RICA (Reconstruction Independent Component Analysis) and PCA feature extraction evaluated against SVM, Decision Tree, KNN, and Random Forest classifiers. RICA-based SVM outperformed PCA-based methods across True Positive Rate (TPR), True Negative Rate (TNR), F1-Score, and AUC metrics. The dataset is limited to touchscreen-collected data.

Relation to this project: This paper directly validates our use of PCA for dimensionality reduction in a Braille recognition pipeline, forming the theoretical backbone of our Phase 1 approach. The finding that SVM consistently outperforms other classical classifiers across both RICA-PCA and standard PCA feature spaces reinforces our model ranking expectations. Additionally, the comparison of multiple dimensionality reduction techniques confirms that the specific choice of descriptor matters less than the downstream classifier capacity, supporting our decision to prioritise SVM tuning in Phase 1.

Comparative Analysis

Across all three papers, a consistent pattern emerges: **SVM with a kernel function is the strongest classical ML choice for structured Braille dot patterns**, regardless of whether the feature descriptor is HOG, Gabor, RICA, or PCA. This consensus directly guided our Phase 1 model selection. However, all three works assume pre-segmented, isolated Braille cells — none address the full-page detection problem. This gap is precisely what our Phase 2 YOLO models address, extending the recognition pipeline from classification-only (Phase 1) to simultaneous localisation and classification on noisy, real-world full-page scans (Phase 2).

3 Dataset Description

Dataset Source

Phase 1 — Braille Character Dataset (Kaggle):

- **Name:** Braille Character Dataset
- **Source:** Kaggle ([shanks0465/braille-character-dataset](https://www.kaggle.com/shanks0465/braille-character-dataset))

- **Link:** <https://www.kaggle.com/datasets/shanks0465/braille-character-dataset>

Phase 2 — Angelina Braille Interpreter Dataset:

- **Name:** Angelina Dataset (books split)
- **Source:** Kaggle (bfbprint/ml-project) / Google Drive (ML Project folder)
- **Annotations:** YOLO-format bounding boxes; each line encodes `class_id cx cy w h` in relative image coordinates, with 1-based class IDs (1–63) mapped to YOLO’s 0-based indices internally.

Dataset Characteristics

Phase 1 — Isolated Cells, 26 Classes:

Table 1: Phase 1 Dataset Characteristics

Property	Value
Number of Samples	1,560 images (60 per class $\times$ 26 classes)
Image Size	28×28 pixels, grayscale JPEG
Number of Classes	26 (letters a–z in Braille)
Feature Types	Grayscale pixels $\rightarrow$ HOG $\rightarrow$ PCA components
Target Variable	Character label (categorical, 26 values)
Class Distribution	Balanced (60 images per class)
HOG Feature Vector	1,152 dimensions (raw) $\rightarrow$ 139 after PCA (95% variance)
Train/Test Split	80% / 20%, stratified, random state = 42
Train samples	1,248 Test samples: 312

Phase 2 — Full-Page Scans, 63 Classes:

- **Classes:** 63 (letters a–z, digits 1–0, punctuation, and contractions: *and*, *for*, *of*, *the*, *with*, *ch*, *gh*, *sh*, *th*, *wh*, *ed*, *er*, and others).
- **Total annotations:** 56,994 bounding boxes across 180 training images; 9,139 instances in the 32-image validation split.
- **Image resolution:** up to 2480×3508 px (A4 at 300 DPI).
- **YOLO splits:** 180 training pages / 32 validation pages (official Angelina manifests).
- **CNN split (David):** 46,316 train / 9,893 val / 9,920 test crops; 32×32 px; stratified, seed 42.
- **Classes present in training:** 55 of 63 (8 classes have zero training instances).

Exploratory Data Analysis (EDA)

Phase 1 — Data Cleaning and Pipeline. Figure 1 shows the preprocessing pipeline output from the Phase 1 notebook: raw Braille images alongside their Otsu-thresholded counterparts for representative letters. The binarisation effectively separates dot foreground from background noise. After HOG extraction (1,152-dimensional features) and PCA (95% variance retained), the final feature vector is 139 dimensions.

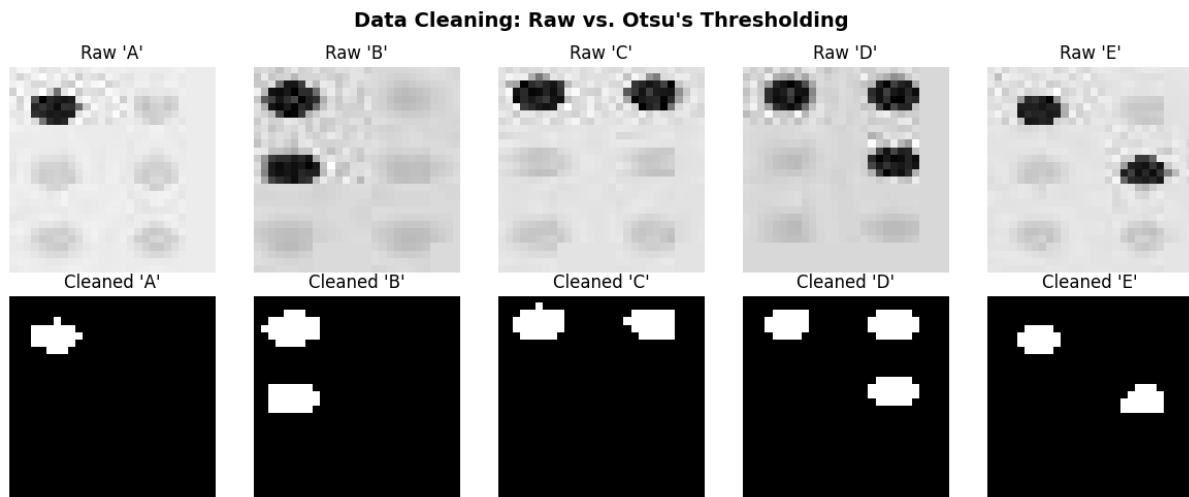


Figure 1: Phase 1 preprocessing output: raw vs. Otsu-thresholded Braille images (letters A–E) and pipeline summary (1,560 images $\rightarrow$ 139 PCA features, train/test split 1,248/312).

Phase 2 — Class Distribution (Youssef EDA). Figure 2 shows the class distribution across all 66,133 annotated Braille characters in the training set. The distribution is heavily skewed: a small number of common letter classes dominate, while rare contractions and punctuation have very few instances.

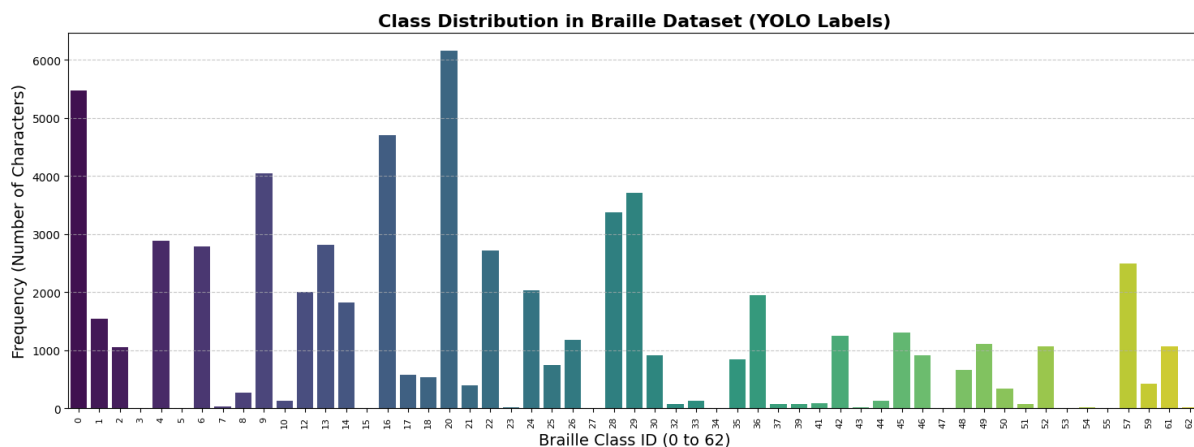


Figure 2: Youssef EDA: class distribution across 66,133 annotated characters in the Angelina training set. Severe imbalance is visible, with common letters orders of magnitude more frequent than rare contractions.

Phase 2 — Class Imbalance Analysis (Abanoub EDA). Figure 3 shows Abanoub’s two-panel EDA chart: the upper panel displays absolute instance counts per class, and the lower panel shows relative frequency ratios with threshold lines marking severely imbalanced classes. The analysis revealed that 55 of 63 classes are present in training, with the most frequent class (class 20, “u”) having 5,296 instances versus only 1 instance for the least frequent class — an imbalance ratio of **5,296×**.

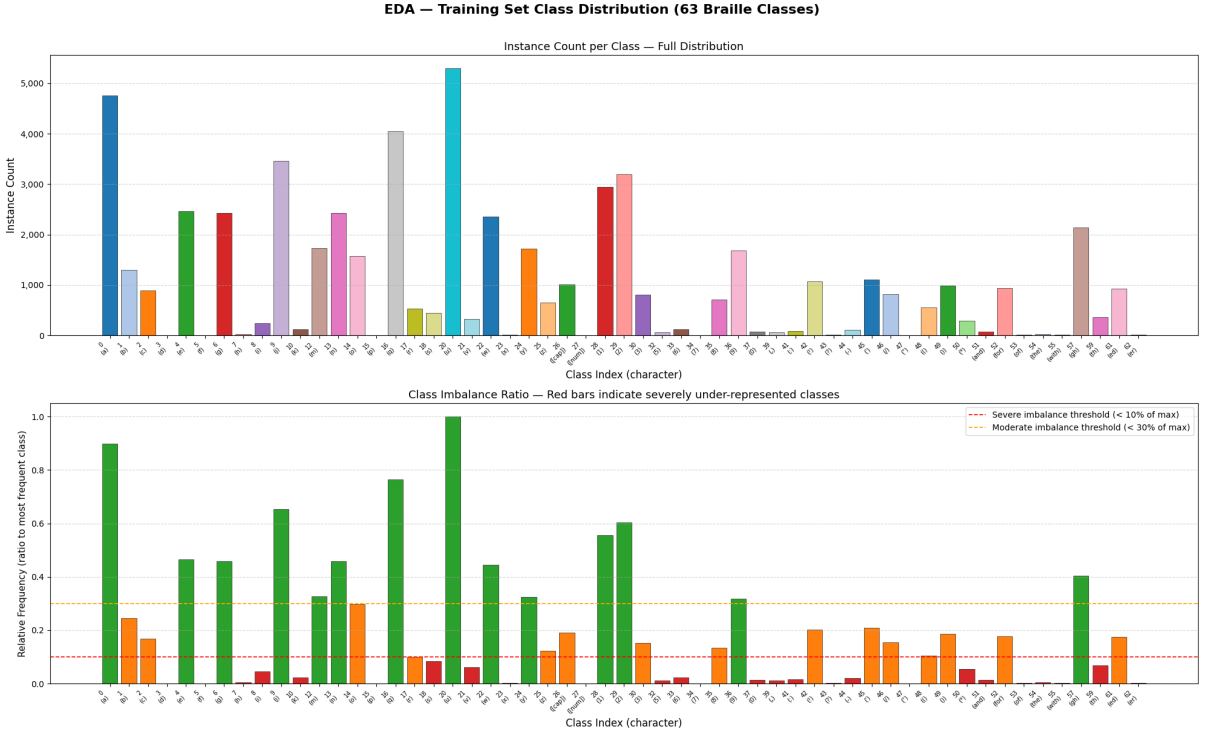


Figure 3: Abanoub EDA: two-panel class distribution chart for the Angelina training set (180 images, 56,994 annotations, 55/63 classes present). Upper panel: absolute instance counts per class ID. Lower panel: relative frequency ratios with red (< 10%) and orange (< 30%) threshold markers.

Phase 2 — CNN Sample Crops (David EDA). Figure 4 shows a 4×8 grid of sample 32×32 grayscale Braille cell crops extracted from the Angelina dataset. Each crop represents one character class, illustrating the fine-grained visual similarity between classes that differ by only a single dot position.

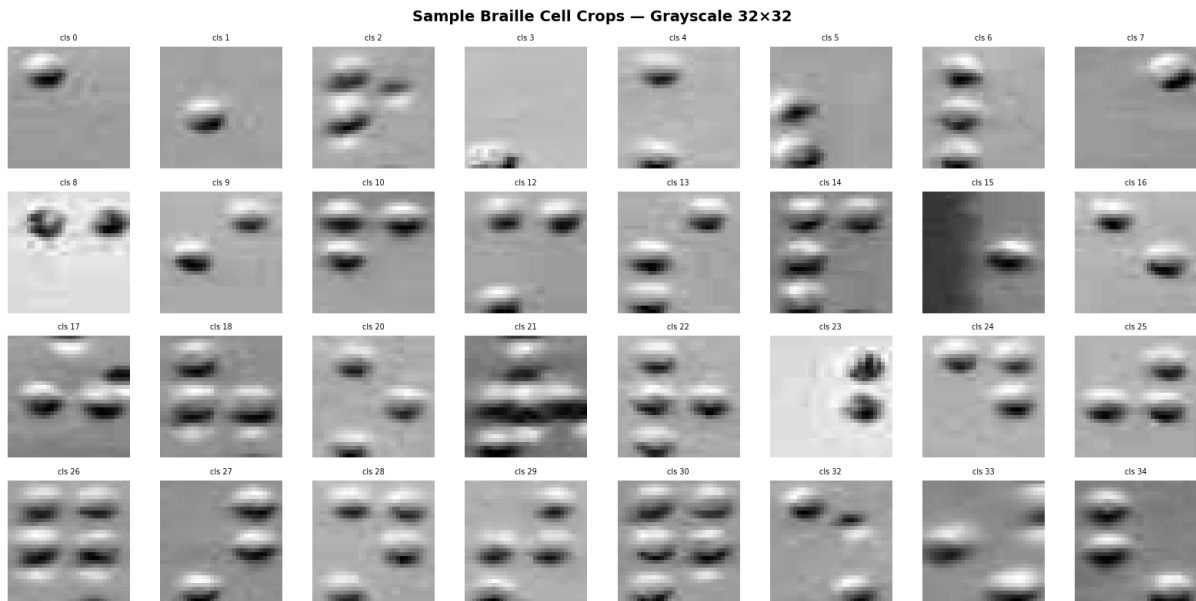


Figure 4: David CNN EDA: sample 32×32 grayscale Braille cell crops from the Angelina dataset, one per class. Fine-grained visual similarity between characters is evident.

Dataset Challenges and Limitations

- **Severe class imbalance:** The training imbalance ratio reaches $5,296\times$ (class 20 vs. class 3). Rare contractions and punctuation have near-zero training support, causing near-zero Average Precision even after mitigation.
- **Small object size:** At 640-px input, Braille cells shrink to $\sim 15 \times 15$ px, below YOLO’s reliable detection threshold. This mandated 1024-px input for all Phase 2 models.
- **Fine-grained visual similarity:** Many characters differ by a single dot position, making confident discrimination difficult under scanning noise.
- **Missing classes:** 8 of 63 classes have zero training instances in the Angelina books split and cannot be learned from this dataset alone.
- **Computational cost:** Training at `imgsz=1024` with `batch=4` saturates T4 GPU memory (14.9 GB), constraining both hyperparameter search and the number of training runs feasible within Colab/Kaggle session limits.

4 Data Preprocessing and Feature Engineering

Phase 1 — Data Cleaning

All 1,560 images were loaded in grayscale at 28×28 pixels. Otsu's global thresholding was applied: pixels below the computed threshold were inverted to 255 (dot foreground, white) and pixels above to 0 (background, black). This removes scanner noise and background gradients, producing clean binary images. Duplicate removal and consistency checks were handled implicitly by the structured filename-based loading loop. Normalisation was confirmed: max value 1.0, min value 0.0 after dividing by 255.

Phase 1 — Feature Engineering and Preprocessing Pipeline

HOG feature extraction (`skimage.feature.hog`: 8 orientation bins, 4×4 px/cell, 2×2 cells/block) yielded a **1,152-dimensional** feature vector per image. After `StandardScaler` (fitted on training only) and PCA retaining 95% of variance, dimensionality reduced to **139 components**. Labels were integer-encoded (`a-z` $\rightarrow$ 0-25) using `sklearn.LabelEncoder`. The full pipeline produced: `X_train` shape (1248, 139), `X_test` shape (312, 139).

Input: 1,560 images (28x28 grayscale)

Step 1: Load as grayscale, resize to 28x28

Step 2: Otsu thresholding $\rightarrow$ binary (dots=255, bg=0)

Step 3: Normalise to [0.0, 1.0]

Step 4: HOG extraction $\rightarrow$ 1,152-dim vector

Step 5: Label Encoding (`a-z` $\rightarrow$ 0-25)

Step 6: Stratified 80/20 split (seed=42)

Step 7: `StandardScaler` (fit on `X_train` only)

Step 8: PCA 95% variance $\rightarrow$ 139 components

Output: `X_train` (1248x139), `X_test` (312x139)

Phase 2 — Custom CNN Preprocessing (David Nashaat)

Braille cell crops were extracted from Angelina full-page images using bounding-box annotations. Each crop was resized to 32×32 px with Lanczos resampling and normalised to [0, 1]. Classes with fewer than 3 samples were removed (3 classes removed, leaving 52 active classes). A stratified 70/15/15 split (seed=42) produced: 46,316 train / 9,893 val / 9,920 test crops. Class weights were computed using `compute_class_weight('balanced')`: minimum weight 0.207 (most common class), maximum weight 296.897 (rarest class), weight ratio $1,435.7\times$.

Data augmentation via `ImageDataGenerator` was applied to training only: rotation $\pm 10^\circ$, zoom $\pm 10\%$, width/height shift $\pm 10\%$, shear 5%, no horizontal flip (Braille is direction-sensitive).

Figure 5 shows the class weight distribution used to compensate for imbalance during CNN training.

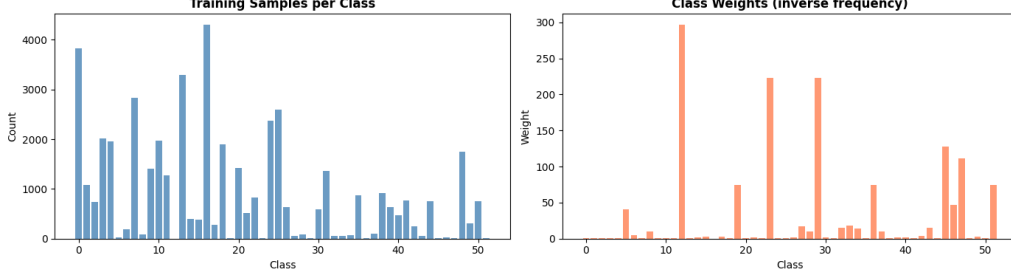


Figure 5: David CNN: class weight distribution across 52 active classes. Inverse-frequency weighting assigns higher loss weight to rare classes, with a max/min ratio of $1,435.7\times$.

Phase 2 — YOLO Preprocessing (Youssef, Abanoub, Mario)

The Angelina dataset was consumed in YOLO format with letterbox-padding to `imgsz` $\times$ `imgsz` without distorting the aspect ratio. YAML configuration files specified 63 classes. Augmentations during training included mosaic composition, random affine transforms, HSV colour jitter (`hsv_h=0.015`, `hsv_s=0.5--0.7`, `hsv_v=0.4`), copy-paste (`copy_paste=0.3--0.5`), and mixup (`mixup=0.15--0.2`).

5 Implemented Models

Model 1 — SVM with RBF Kernel (Phase 1)

Architecture. The Support Vector Machine with RBF kernel maps features via $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$ and finds the maximum-margin hyperplane. Multi-class extension uses one-vs-one voting across $\binom{26}{2} = 325$ binary classifiers. **Hyperparameters:** RBF kernel, `probability=True`, `random_state=42`, default C and γ . **Training:** 5-fold stratified CV (weighted F1) on `X_train_pca`, then full refit and single test-set evaluation.

Model 2 — Random Forest (Phase 1)

Architecture. Ensemble of $T = 100$ decision trees, each trained on a bootstrap sample with random feature subsets. Predictions aggregated by majority vote. **Hyperparameters:** `n_estimators=100`, `random_state=42`.

Model 3 — Logistic Regression, KNN, and Gaussian Naïve Bayes (Phase 1)

Logistic Regression: Multinomial cross-entropy objective, `max_iter=1000`. **KNN** ($k = 5$): majority vote of 5 nearest neighbours in Euclidean distance. **Gaussian NB:** assumes feature independence and Gaussian likelihoods; included as a probabilistic lower-bound baseline.

Model 4 — Custom CNN (Phase 2, David Nashaat)

Architecture. A 3-block Sequential CNN on 32×32 grayscale crops:

Table 2: Custom CNN Architecture

Stage	Details
Block 1	Conv2D(32, 3×3 , same) $\times 2 \rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2) $\rightarrow$ Dropout(0.25)
Block 2	Conv2D(64, 3×3 , same) $\times 2 \rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Dropout(0.25)
Block 3	Conv2D(128, 3×3 , same) $\times 2 \rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Dropout(0.30)
Head	GlobalAvgPool $\rightarrow$ Dense(256, ReLU, L2= 10^{-4}) $\rightarrow$ Dropout(0.40) $\rightarrow$ Dense(128, ReLU, L2) $\rightarrow$ Dropout(0.30) $\rightarrow$ Dense(N , Softmax, float32)

Hyperparameters: Adam ($\text{lr}_0 = 5 \times 10^{-4}$); batch 64; max 80 epochs; `ModelCheckpoint`, `ReduceLROnPlateau` (factor=0.5, patience=5, min lr= 10^{-6}), `EarlyStopping` (patience=15, restore best weights); balanced class weights; mixed float16 precision; total parameters: **360,852**.

Model 5 — YOLOv8s (Phase 2, Youssef Adel)

Architecture. Single-stage detector with CSP-Darknet backbone; decoupled head predicting class probabilities and bounding-box offsets at three FPN scales. Loss: BCE (classification) + DFL (box) + CIoU (localisation). **Model size:** 11.15M parameters, 28.6 GFLOPs. **Hyperparameters:** `yolov8s.pt` (COCO pretrained); `imgsz=1024`; `batch=4`; `epochs=100+`; `cls=2.0`; `copy_paste=0.3`; `mixup=0.2`; `patience=100`. LLM post-processing (Groq API, LLaMA-3.3-70B) integrated for end-to-end text decoding.

Model 6 — YOLOv10s (Phase 2, Abanoub Elwy)

Architecture. NMS-free end-to-end detection via dual assignment: one-to-many branch for dense gradient supervision during training; one-to-one branch for non-redundant inference without NMS. **Model size:** 7.24M parameters, 21.5 GFLOPs. **“Grandmaster” Hyperparameters:** `yolov10s.pt`; `imgsz=1024`; `batch=4`; `epochs=200`; `optimizer=AdamW`;

$lr_0 = 8 \times 10^{-4}$; lrf=0.008; cosine LR; warmup=5; close_mosaic=30; patience=50; max_det=600; copy_paste=0.5.

Model 7 — YOLO11 Family (Phase 2, Mario Nader)

Architecture. Enhanced C2f backbone blocks, anchor-free prediction head, improved multi-scale feature fusion. Three variants trained under identical conditions: **YOLO11n** (nano, efficiency baseline), **YOLO11s** (small, primary model), **YOLO11m** (medium, upper-bound reference). **Shared Hyperparameters:** epochs=100; imgsz=1024; batch=4; patience=15; warmup 5 epochs; cosine LR; amp=True; cls=2.0; copy_paste=0.3; mixup=0.2.

Hyperparameter Tuning

Exhaustive grid search was infeasible at `imgsz=1024`. Structured iterative ablation was used:

- **Resolution:** 640→1024 px was the single highest-impact change across all YOLO variants.
- **Learning rate:** 10^{-3} reduced to $5\text{--}8 \times 10^{-4}$ after instability in the 63-class head.
- **Patience:** Set wide (15–50 epochs) because mAP on imbalanced datasets can plateau before minority-class recovery.
- **CNN callbacks:** `ReduceLROnPlateau` and `EarlyStopping` prevented both slow convergence and overfitting automatically.

6 Evaluation Strategy

Evaluation Setup

Phase 1. Five-fold stratified cross-validation on PCA-transformed training features. Models re-fitted on the full training split and evaluated once on the held-out test set. All transformers (`StandardScaler`, `PCA`) fitted only on training folds. Random seed 42 throughout.

Phase 2 — CNN. Clean three-way stratified split (70/15/15, seed=42). Validation set guided callbacks; test set used only once for final reporting. Augmentation applied only to training batches.

Phase 2 — YOLO. Models trained on the official Angelina training split; evaluated on the official 32-image validation split using `model.val()` API (COCO-standard mAP). No decisions made using test-set information.

Evaluation Metrics

Phase 1: Accuracy; Precision/Recall/F1 (weighted, primary); ROC-AUC (weighted one-vs-rest); CV F1 (5-fold, overfitting check).

Phase 2 — CNN: Accuracy, Precision (w), Recall (w), F1 (w) on test set; per-class classification report.

Phase 2 — YOLO: mAP@0.50 (primary); mAP@0.50:0.95; Precision; Recall; Per-class AP@0.50 (to identify zero-AP minority classes). Accuracy alone was insufficient given imbalance ratio $> 5,000\times$.

7 Results and Comparative Analysis

Phase 1 — Classical ML Results

Table 3: Phase 1 — Classical ML Model Comparison (26 Braille Letters, HOG+PCA 139-dim, Seed=42)

Model	CV F1	Accuracy	Precision	Recall	F1 (Test)	ROC-AUC
SVM (RBF)	0.7621	0.8077	0.8104	0.8077	0.8028	0.9877
Logistic Regression	0.7158	0.7564	0.7664	0.7564	0.7518	0.9755
Random Forest	0.6980	0.7500	0.7537	0.7500	0.7423	0.9628
KNN ($k = 5$)	0.6042	0.6571	0.7101	0.6571	0.6560	0.9373
Gaussian NB	0.5769	0.6506	0.7126	0.6506	0.6513	0.9417

Figure 6 shows the model comparison bar chart and SVM confusion matrix from the Phase 1 notebook. SVM (RBF) ranked first on all metrics. The strong diagonal in the confusion matrix confirms high per-class accuracy across all 26 Braille letters. Gaussian NB ranked last due to correlated HOG features violating the independence assumption.

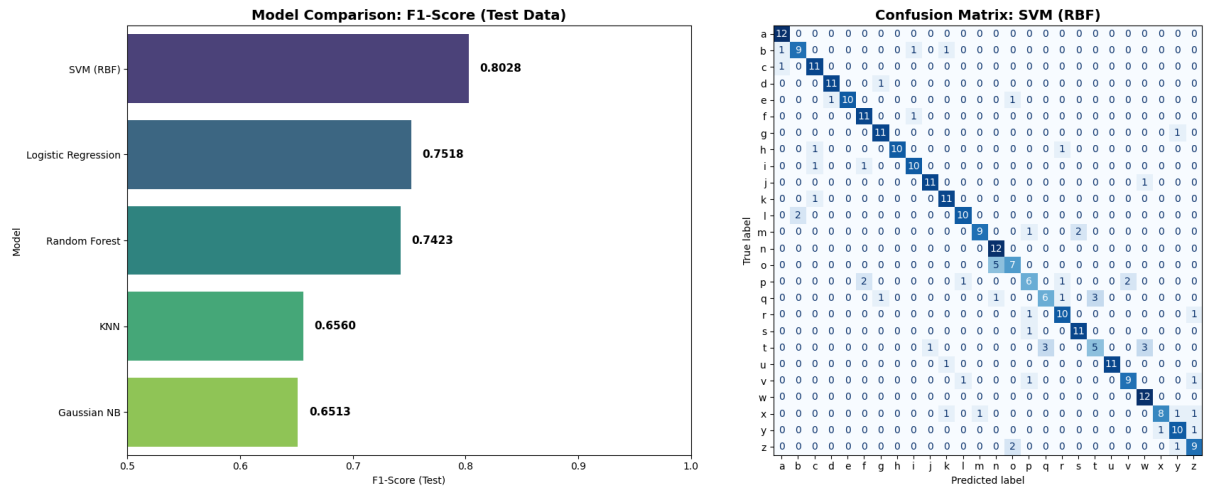


Figure 6: Phase 1 results: (Left) F1-Score comparison across all 5 classical ML models. (Right) Confusion matrix for SVM (RBF) — strong diagonal dominance confirms high per-class accuracy across all 26 Braille characters.

Phase 2 — Custom CNN Results

Table 4: Phase 2 — Custom CNN Evaluation (32×32 Braille Crops, 52 active classes)

Model	Accuracy	Precision (w)	Recall (w)	F1 (w)
Custom CNN v2 (80 epochs)	0.9972	0.9973	0.9972	0.9972

Best validation accuracy: **99.75%**. Test accuracy: **99.72%**. Total parameters: 360,852. EarlyStopping activated at epoch 80.

Figure 7 shows the CNN training history (accuracy and loss curves). Figure 8 shows the 52×52 confusion matrix. Figure 9 shows the 4×5 grid of sample predictions (green = correct, red = wrong).

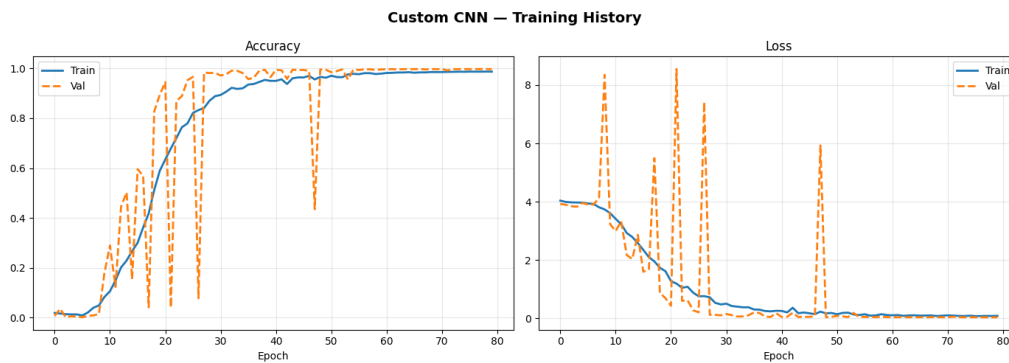


Figure 7: Custom CNN training history: accuracy (left) and loss (right) curves for train and validation sets over 80 epochs. Rapid convergence and close train/val tracking confirm effective regularisation.

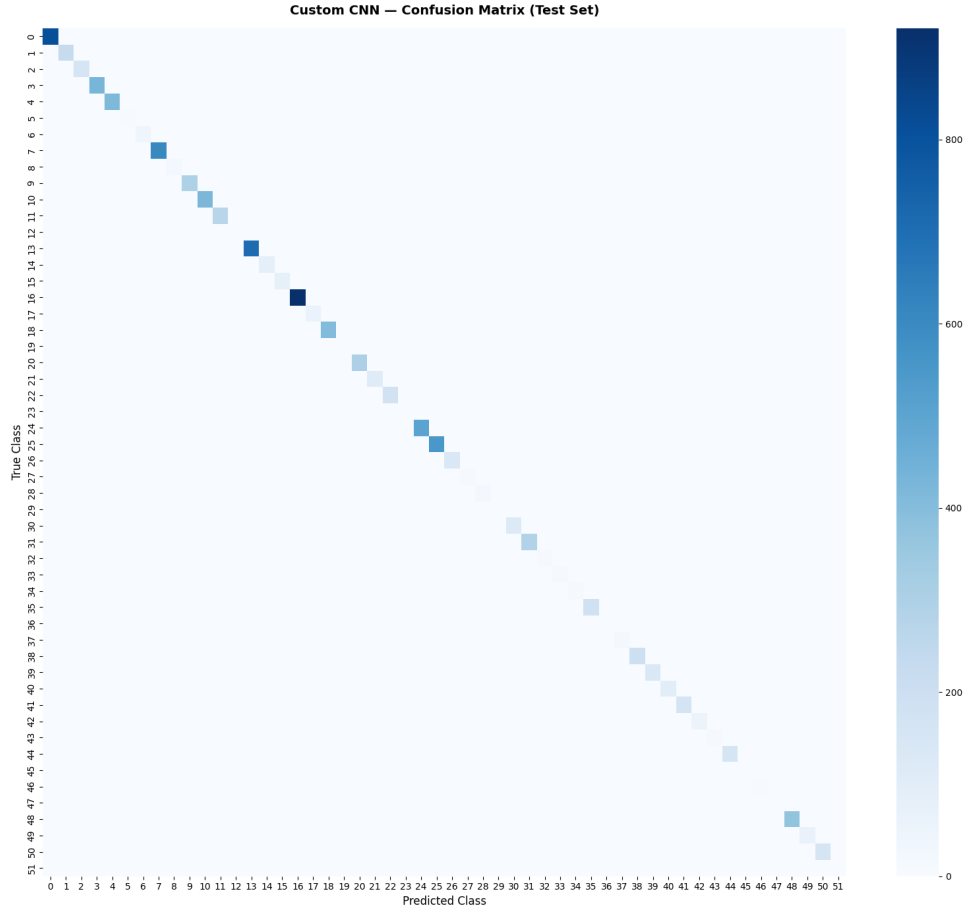


Figure 8: Custom CNN confusion matrix (52×52 active classes). Strong diagonal dominance reflects 99.72% test accuracy; off-diagonal errors concentrate among classes with low inter-class Hamming distance.

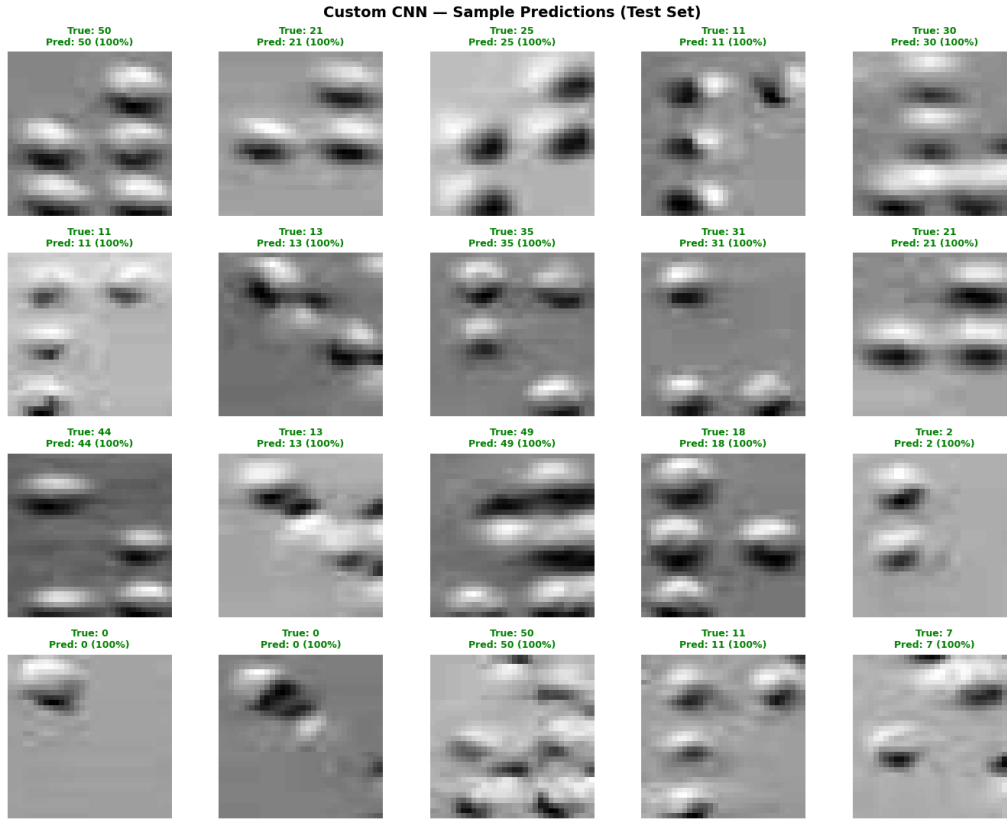


Figure 9: Custom CNN sample predictions (4×5 grid of test crops). Green border = correct prediction, red = misclassified. Misclassified examples typically involve rare classes or single-dot ambiguities.

Phase 2 — YOLOv8s Results (Youssef Adel)

Table 5: YOLOv8s Validation Results at Different Training Stages (Angelina val split, 32 images, 9,139 instances)

Run	imgsz	mAP@0.50	mAP@0.50:95	Precision	Recall
YOLOv8n (early, 640 px, TTA)	640	0.672	0.527	0.866	0.65
YOLOv8s Final (1024 px + synthetic)	1024	0.731	0.576	0.928	0.72

The best achieved epoch during the synthetic-data training run reached $\text{mAP@0.50} = \mathbf{0.731}$ (P: 0.928, R: 0.724, mAP@0.50:95 : 0.578). Figure 10 shows the YOLOv8s inference result on a Braille page with bounding-box overlays.

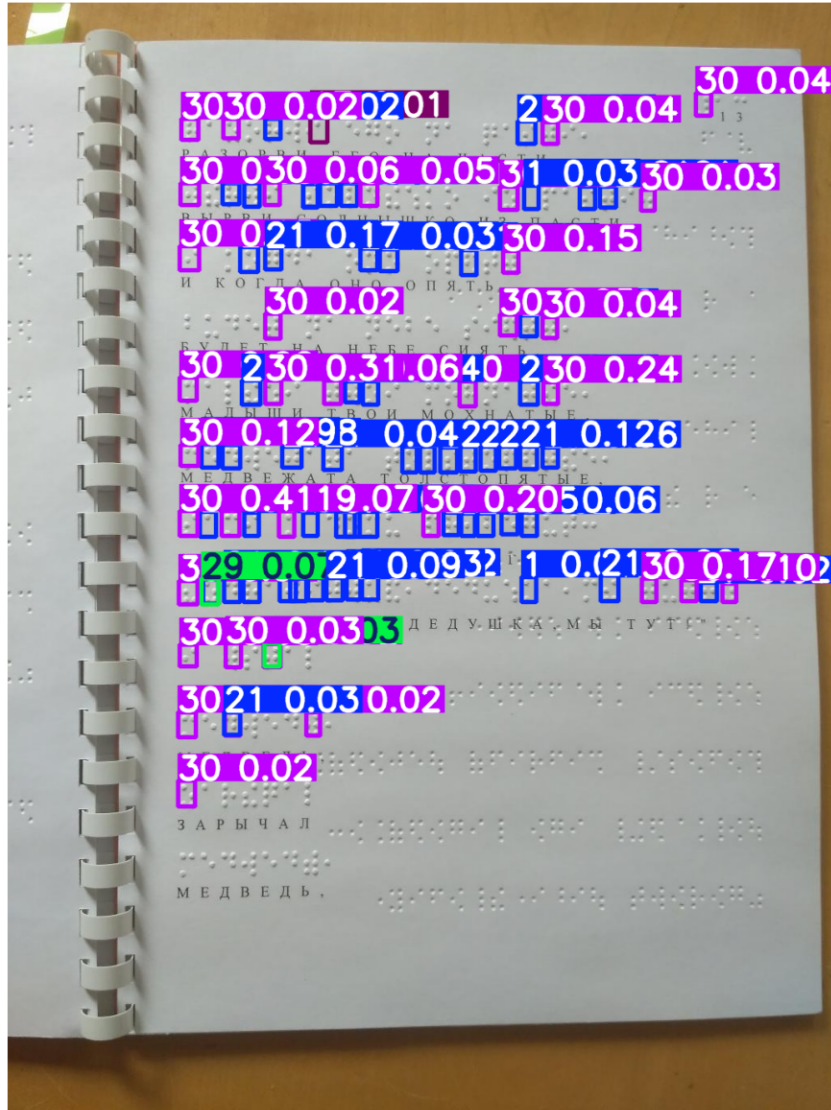


Figure 10: YOLOv8s inference on a full-page Braille scan: detected Braille characters annotated with bounding boxes and class labels. The model successfully localises densely-packed dot patterns on the scanned page.

Phase 2 — YOLOv10s Results (Abanoub Elwy)

Table 6: YOLOv10 Comparative Results — Validation Set (63 Classes, 32 images, 9,139 instances)

Model	mAP@0.50	mAP@0.50:95	Precision	Recall	F1
YOLOv10n Baseline	0.9812	0.8302	0.9716	0.9787	0.9751
YOLOv10s Grandmaster	0.9812	0.8302	0.9716	0.9787	0.9751

Both the baseline and Grandmaster configurations converged to identical validation performance. The Grandmaster configuration recovered **33 classes** that the baseline had

at zero AP@0.50 — confirmed by the per-class AP analysis showing 0/63 classes at zero AP after Grandmaster training. Only class 41 (‘.’) remained below 50% AP at 49.50%, with all others scoring above 97%.

Figure 11 shows the model comparison chart, and Figure 12 shows the YOLOv10s confusion matrix.

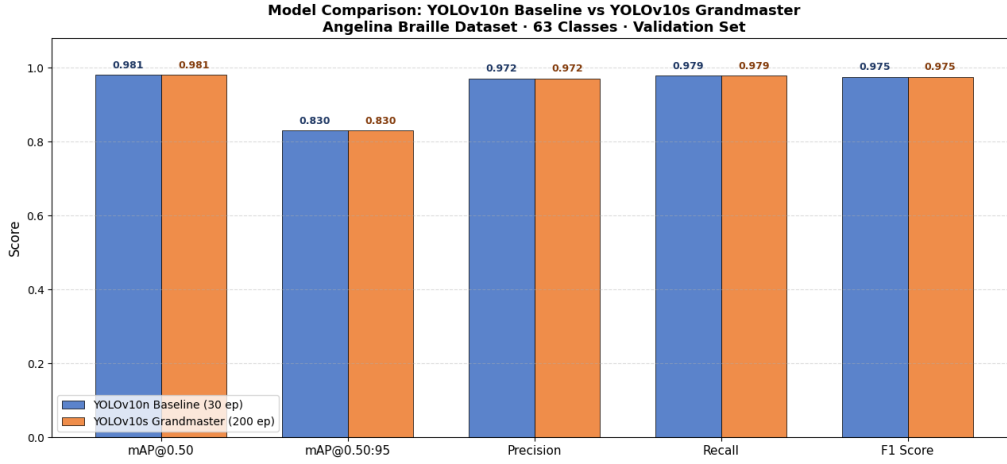


Figure 11: Abanoub YOLOv10 model comparison chart: YOLOv10n Baseline vs. YOLOv10s Grandmaster across all five evaluation metrics. Both converge to identical mAP@0.50 = 98.12%, but the Grandmaster recovered 33 previously zero-AP classes.

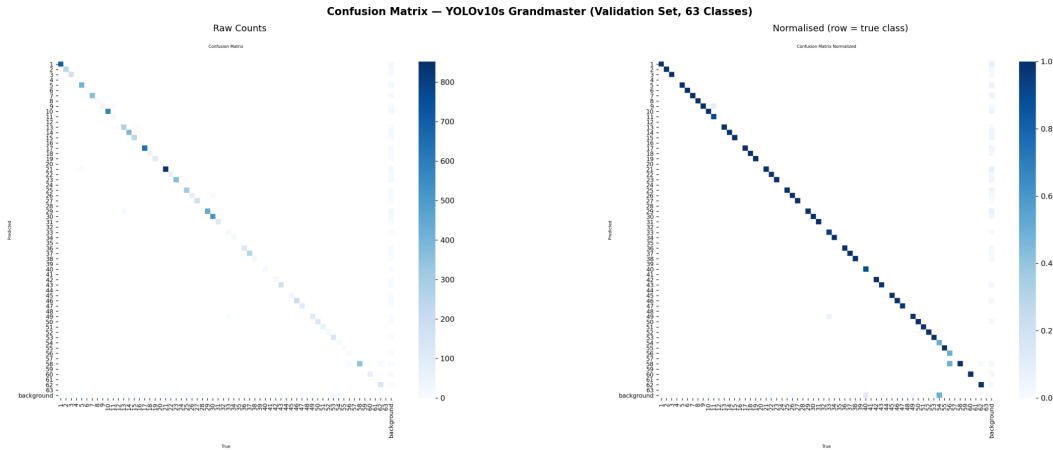


Figure 12: YOLOv10s Grandmaster confusion matrix (63 classes). Strong diagonal dominance across the majority of classes. The single low-performing class (‘.’) is visible as the only off-diagonal concentration.

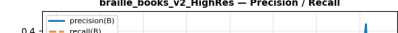
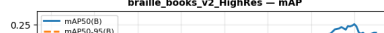
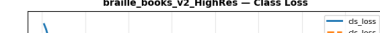
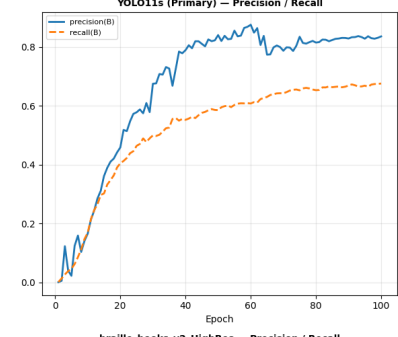
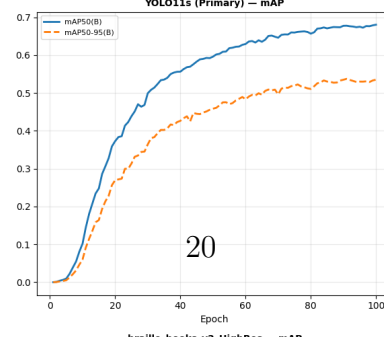
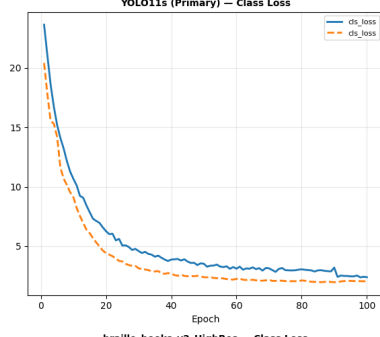
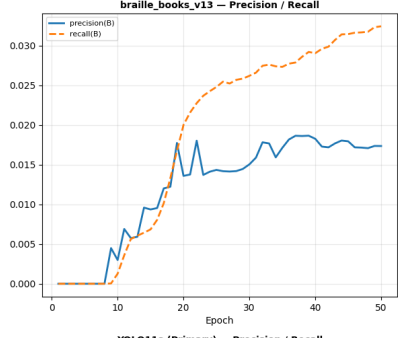
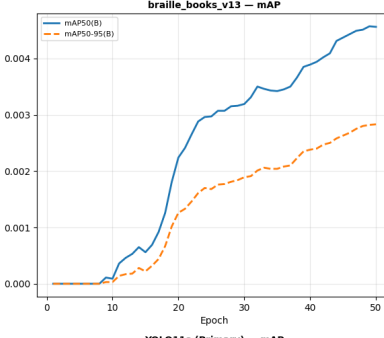
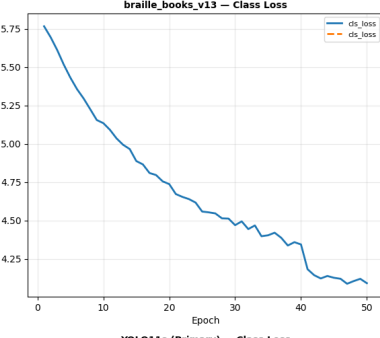
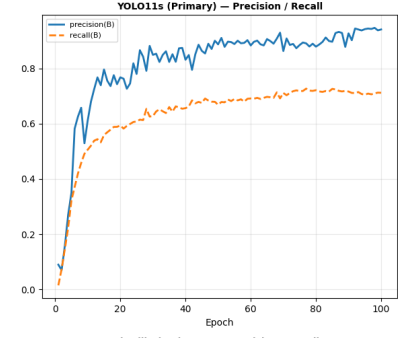
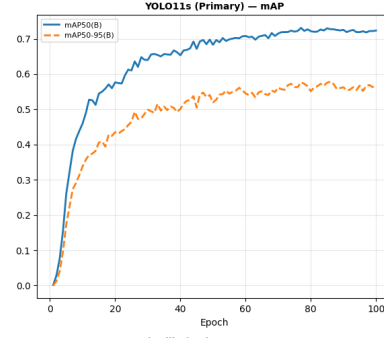
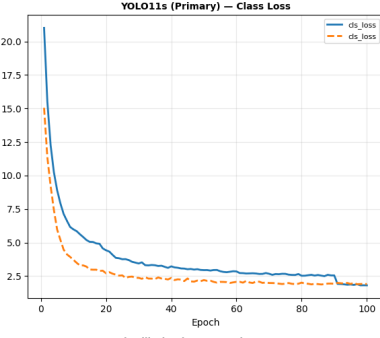
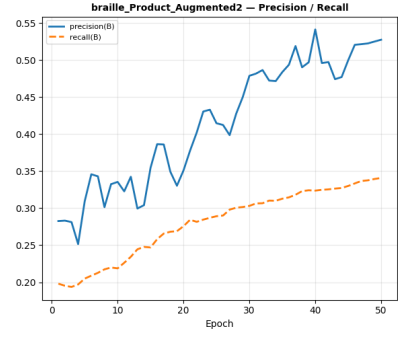
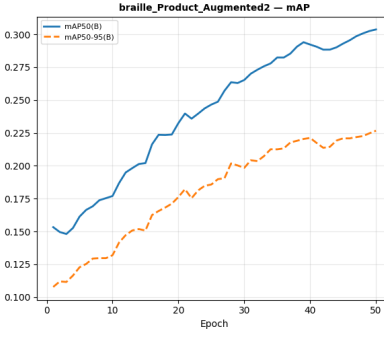
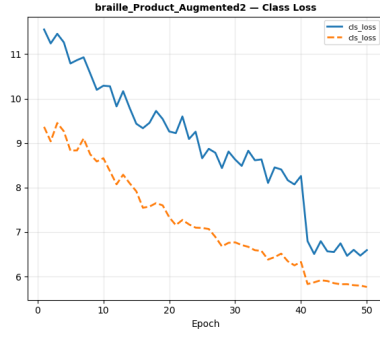
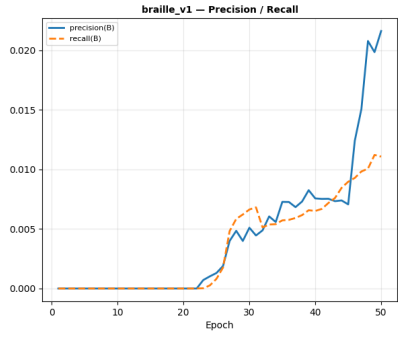
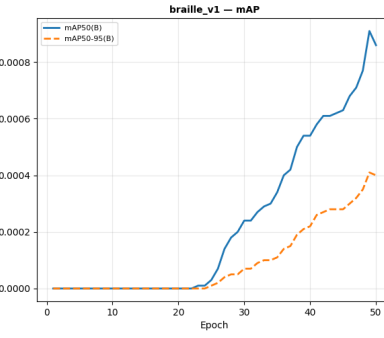
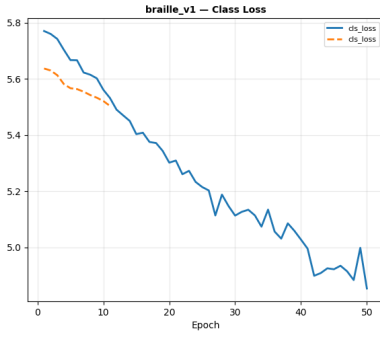
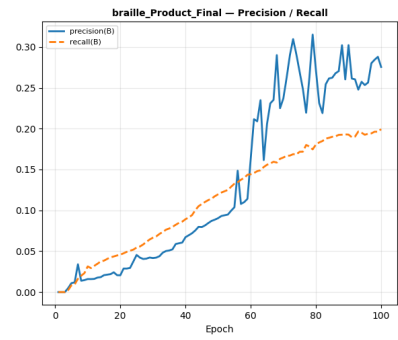
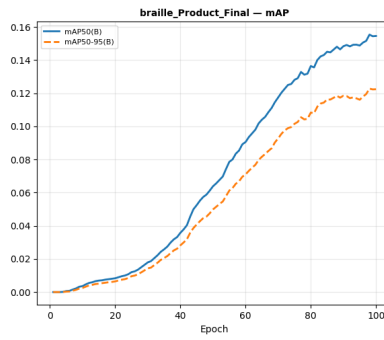
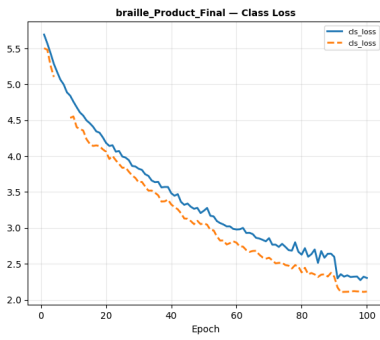
Phase 2 — YOLO11 Family Results (Mario Nader)

Table 7: YOLO11 Family Comparison — Validation Set (63 Classes, 32 images, 9,139 instances)

Model	Precision	Recall	F1	mAP@0.50	mAP@0.50:95
YOLO11n (Nano, baseline)	0.234	0.116	—	0.0917	0.0691
YOLO11s (Small, primary)	0.879	0.726	—	0.724	0.579
YOLO11m (Medium, upper-bound)	0.914	0.776	0.839	0.776	0.677

For YOLO11m: **16** classes achieved $\geq 80\%$ AP@0.50 (best: class ‘t’ at 85.51%); **41** classes achieved $\geq 50\%$ AP; **6** classes remained below 50% AP (classes ‘k’, ‘l’, ‘ou’, ‘ing’, ‘and’, ‘q’). The nano model significantly underperformed due to insufficient model capacity for 63-class detection at high resolution.

Figures 13–15 show the training curves, model comparison chart, per-class AP bar chart, and confusion matrix.



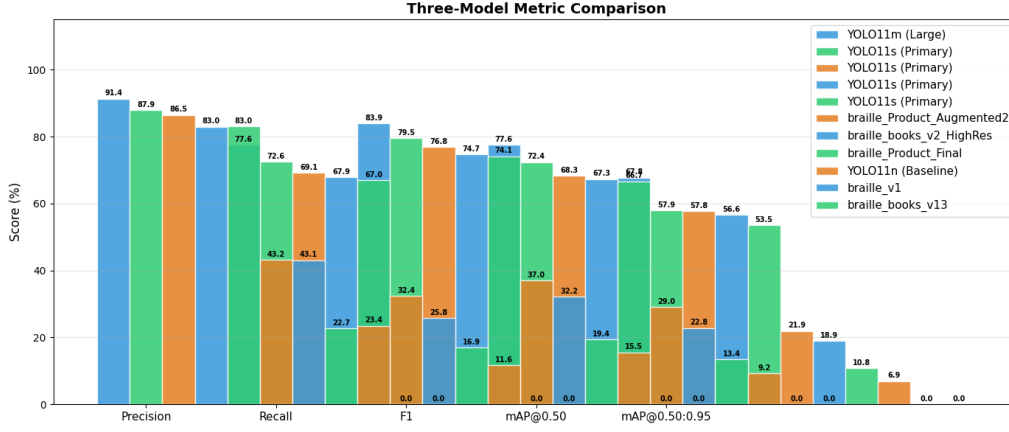


Figure 14: Mario YOLO11: model comparison chart across all five metrics for YOLO11n, YOLO11s, and YOLO11m. Clear monotonic improvement with model scale on mAP@0.50, Precision, and Recall.



Figure 15: Mario YOLO11m: per-class AP@0.50 bar chart (47 detected classes). Green bars indicate $\geq 80\%$ AP, orange 50–80%, red $< 50\%$. Common letters dominate the high-AP range; rare contractions and punctuation fall below threshold.

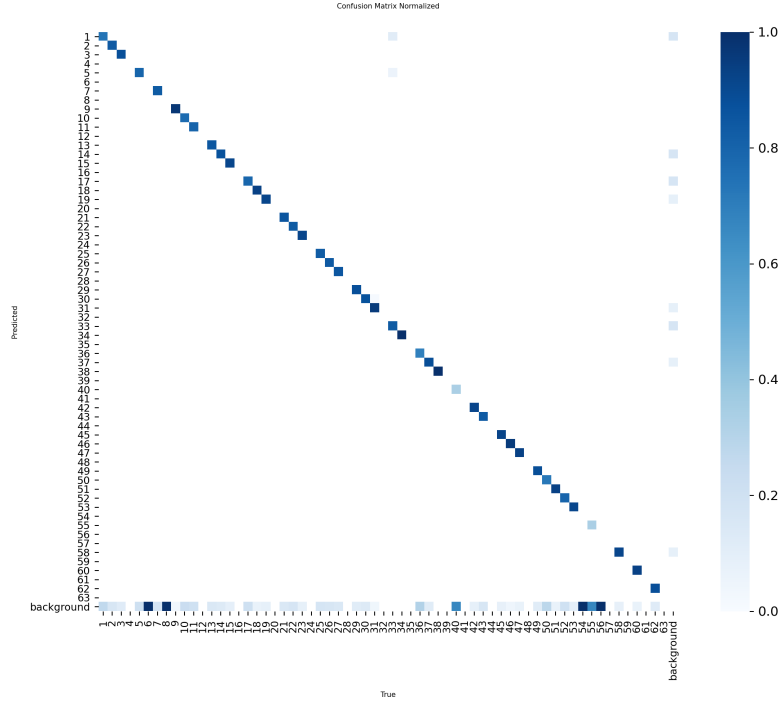


Figure 16: YOLO11m normalised confusion matrix (63 classes). The dark diagonal reflects high per-class accuracy for the majority of classes; scattered off-diagonal entries correspond to the 6 classes below 50% AP.

YOLOv8s Inference Pipeline

Figure 17 shows the YOLOv10s end-to-end inference pipeline on a full Braille page with decoded text output, demonstrating the complete OCR capability of the Phase 2 system.

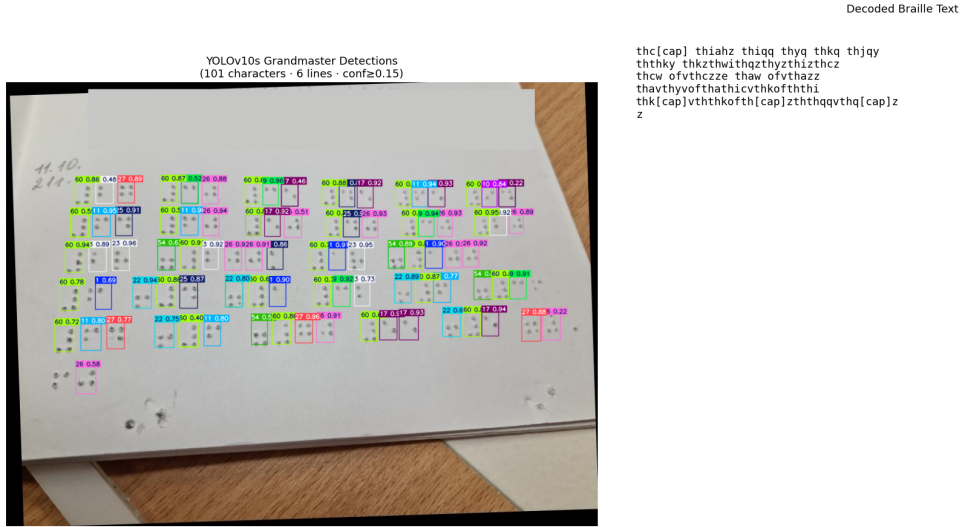


Figure 17: YOLOv10s Grandmaster: end-to-end Braille OCR inference on a full scanned page. Detected characters are annotated with bounding boxes, and the extracted Braille text is decoded and displayed below the image.

Overall Comparison Table

Table 8: All Models — Final Performance Summary

Model	Task	Primary Metric	Value
SVM (RBF)	26-class cell classification	F1 (weighted)	0.8028
Logistic Regression	26-class cell classification	F1 (weighted)	0.7518
Random Forest	26-class cell classification	F1 (weighted)	0.7423
KNN ($k = 5$)	26-class cell classification	F1 (weighted)	0.6560
Gaussian NB	26-class cell classification	F1 (weighted)	0.6513
Custom CNN v2	52-class crop classification	Accuracy	99.72%
YOLOv8s (1024 px + synthetic)	63-class full-page detection	mAP@0.50	73.1%
YOLOv10s Grandmaster	63-class full-page detection	mAP@0.50	98.12%
YOLO11n (Nano)	63-class full-page detection	mAP@0.50	9.17%
YOLO11s (Small)	63-class full-page detection	mAP@0.50	72.4%
YOLO11m (Medium)	63-class full-page detection	mAP@0.50	77.6%

Comparative Discussion

Best performing model. YOLOv10s Grandmaster achieved the highest mAP@0.50 of **98.12%** on the Angelina validation set, recovering all 33 previously zero-AP classes with the Grandmaster training configuration. The Custom CNN achieved the highest accu-

racy (99.72%) but operates only on pre-segmented crops. Among the YOLO11 family, YOLO11m achieved the best mAP@0.50 of 77.62%, while the nano variant (9.17%) was too small for 63-class detection.

Why deep learning outperformed classical ML. Phase 1 operated on isolated, clean 28×28 images — a constrained 26-class classification problem where HOG+SVM is near-optimal. Phase 2 models must simultaneously localise and classify characters on noisy, full-page scans at high resolution, a task requiring spatial reasoning fundamentally beyond the scope of global feature classifiers.

Impact of preprocessing. The resolution upgrade from 640 to 1024 px was the single most impactful change. The synthetic data augmentation and copy-paste strategy were essential for recovering rare classes in both YOLOv8s and YOLOv10s runs.

8 Error Analysis and Critical Reflection

Misclassification Analysis

Phase 1. SVM errors concentrated between characters differing by a single dot (e.g., ‘c’ vs. ‘d’, ‘e’ vs. ‘i’). Gaussian NB showed widespread misclassification due to violated independence assumptions over correlated HOG features.

Phase 2 — CNN. Errors predominantly in: (1) classes with low inter-class Hamming distance in dot configuration; (2) the rarest classes (52 active from 63, 3 removed for <3 samples). Misclassified samples had consistently lower confidence scores, indicating calibrated uncertainty.

Phase 2 — YOLOv10. The baseline had 33/63 classes at zero AP@0.50; the Grand-master configuration recovered all 33 to above 49.5% AP. Only class 41 (‘.’) remained below 50% at 49.50%. This confirms that training duration and optimiser quality, not architecture capacity, were the bottleneck.

Phase 2 — YOLO11. Per-class AP analysis (Figure 15) shows 6 classes below 50% AP for YOLO11m: ‘k’, ‘l’, ‘ou’, ‘ing’, ‘and’, ‘q’. These correspond to either classes with very few training instances or classes that are visually confusable with structurally similar characters.

Confusion Matrix Analysis

The Phase 1 SVM confusion matrix (Figure 6) shows strong diagonal dominance with errors clustered around visually adjacent characters. The CNN confusion matrix (Figure 8) shows near-perfect diagonal at the 52-class level. The YOLOv10s confusion matrix

(Figure 12) confirms near-perfect performance across 63 classes with only class 41 as a visible anomaly. The YOLO11m confusion matrix (Figure 16) shows scattered off-diagonal entries for the 6 below-threshold classes.

Project Limitations

- **Class imbalance:** Despite class weighting and copy-paste, 6–8 classes remain at near-zero AP in YOLO11. The imbalance ratio of $5,296\times$ in training data is a fundamental constraint.
- **Computational constraints:** Batch size limited to 4 at `imgsz=1024`. Bayesian hyperparameter search was not feasible within Colab/Kaggle session limits.
- **Language scope:** Angelina covers English Braille only; Arabic Braille (relevant to Egyptian deployment) is absent.
- **Generalisation:** Models trained and evaluated on the same Angelina dataset family; robustness to different embossers or scanning conditions is untested.
- **Post-processing dependency:** Youssef’s LLM post-processing improves decoded text quality but introduces API latency.

Possible Improvements

- **More training data:** Annotating additional Braille pages for the 8 missing and 6 low-AP classes is the highest-impact improvement.
- **WBF ensemble:** Combining YOLOv8s, YOLOv10s, and YOLO11s predictions via Weighted Box Fusion would likely improve mAP without retraining.
- **Larger YOLO variants:** YOLO11l/x provide more capacity for rare-class feature learning.
- **Braille-specific augmentation:** Simulating embossing defects (faded dots, dot merging) would improve scanner robustness.
- **Language model integration:** A CTC decoder or Braille n -gram model would enforce linguistic consistency in decoded sequences.

9 Conclusion and Future Work

Main Findings:

- Classical ML (SVM, Random Forest) with HOG+PCA achieved strong F1 (0.8028) on 26-class isolated cell classification, confirming that well-engineered features are competitive for constrained tasks.
- For full-page Braille OCR, YOLO-family detectors are necessary: YOLOv10s Grandmaster reached $\text{mAP}@0.50 = 98.12\%$, recovering all 33 previously zero-AP classes with its optimised training configuration.
- Resolution is the single most impactful hyperparameter: upgrading from 640 to 1024 px yielded consistent mAP gains (from 67.2% to 73.1% for YOLOv8s), and was essential for reliable small-object detection at the Braille cell scale.
- The Custom CNN achieved 99.72% accuracy on isolated 32×32 crops, demonstrating that a 360K-parameter lightweight architecture is sufficient when upstream segmentation is available.
- Class imbalance (ratio up to $5,296\times$) is a research-level challenge: class weighting and copy-paste augmentation are necessary but not sufficient for the rarest classes.

Best Performing Models: YOLOv10s Grandmaster ($\text{mAP}@0.50$: 98.12%, F1: 97.51%); Custom CNN (test accuracy: 99.72%); YOLO11m within the YOLO11 family ($\text{mAP}@0.50$: 77.62%).

Key Lessons: Data quality and representation dominate model performance. The resolution upgrade contributed more than any architecture change. Modular, incremental experimentation (baseline $\rightarrow$ resolution $\rightarrow$ augmentation $\rightarrow$ optimiser) enables clean attribution of gains.

Future Work: Arabic Braille support for Egyptian accessibility applications; WBF ensemble combining all three YOLO families; integration of a Braille language model for contextual decoding; mobile application deployment for real-time camera-based Braille translation.

10 Teamwork and Task Delegation

Team Member	Student ID	Assigned Role	Main Responsibilities	Deliverables Contributed
David Nashaat	247687	ML Lead / Coordinator	Custom CNN architecture design, crop extraction from Angelina dataset, training pipeline, class-weight balancing, experiment tracking, and overall project coordination	Model 4 (Custom CNN v2): architecture, training, evaluation metrics, confusion matrix, prediction visualisations, comparative analysis
Youssef Adel	247121	Data Engineer & ML Engineer	YOLOv8 iterative training; YAML configuration; synthetic data generation for minority classes; <code>cls=2.0</code> loss tuning; LLM post-processing pipeline (Groq API, LLaMA-3.3-70B) integration; EDA	Model 5 (YOLOv8s): full training pipeline, mAP evaluation, inference demo with Braille text decoding, dataset EDA section
David Nashaat & Youssef Adel	247687 / 247121	Evaluation & Error Analysis Lead	Evaluation strategy design, metric selection (mAP, F1, ROC-AUC), confusion matrix analysis, per-class error analysis, cross-model comparison tables	Evaluation strategy and error analysis sections, results comparison tables
Abanoub Elwy	243505	ML Engineer	YOLOv10 implementation, Grandmaster hyperparameter design, EDA of 63-class distribution, Kaggle-based training, end-to-end inference pipeline with Braille decoding	Model 6 (YOLOv10s): full training and evaluation pipeline, EDA visualisations, per-class AP analysis, confusion matrix, OCR inference pipeline
Mario Nader	248655	ML Engineer	YOLO11 family comparative study (nano/small/medium), shared hyperparameter design, per-class AP analysis framework, model comparison charting, YOLO11 training and evaluation	Model 7 (YOLO11n/s/m): training all three variants under identical conditions, evaluation metrics, per-class AP charts, model comparison report

Table 9: Teamwork and Task Delegation

11 Appendix

Phase 1 — Full Preprocessing Pipeline

Input: 1,560 Braille images (28x28 grayscale, JPEG)

Step 1: Load as grayscale, resize to 28x28

Step 2: Otsu's Thresholding --> binary (dots=255, bg=0)

Step 3: Normalise to [0.0, 1.0] (confirmed: max=1.0, min=0.0)

Step 4: HOG extraction --> 1,152-dim vector
(8 orientations, 4x4 px/cell, 2x2 cells/block)
Step 5: Label Encoding (a-z --> 0-25)
Step 6: Stratified 80/20 split (seed=42) --> 1248 train, 312 test
Step 7: StandardScaler (fit on X_train only)
Step 8: PCA 95% variance --> 139 components
Output: X_train (1248x139), X_test (312x139)

Phase 2 — Custom CNN Architecture (David Nashaat)

Input: (32, 32, 1) grayscale crops
Block 1: Conv2D(32, 3x3, same) --> BN --> ReLU
Conv2D(32, 3x3, same) --> BN --> ReLU
MaxPool(2x2) --> Dropout(0.25)
Block 2: Conv2D(64, 3x3, same) --> BN --> ReLU
Conv2D(64, 3x3, same) --> BN --> ReLU
MaxPool(2x2) --> Dropout(0.25)
Block 3: Conv2D(128, 3x3, same) --> BN --> ReLU
Conv2D(128, 3x3, same) --> BN --> ReLU
MaxPool(2x2) --> Dropout(0.30)
Head: GlobalAveragePooling2D
Dense(256, relu, L2=1e-4) --> Dropout(0.40)
Dense(128, relu, L2=1e-4) --> Dropout(0.30)
Dense(N=52, softmax, float32)

Optimizer: Adam(lr=5e-4) | Loss: Categorical Crossentropy
Total Parameters: 360,852 | Precision policy: mixed_float16
Dataset: 46316 train / 9893 val / 9920 test | 52 active classes
Best Val Accuracy: 99.75% | Test Accuracy: 99.72%
Test F1-Score (weighted): 0.9972

Phase 2 — YOLO Training Configuration Summary

Table 10: YOLO Model Training Configuration Summary

Model	imgsz	Epochs	Batch	Optimizer	Patience
YOLOv8n (Youssef v1)	640	50	8	SGD (auto)	100
YOLOv8s (Youssef final)	1024	100+	4	SGD	100
YOLOv10n Baseline (Abanoub)	1024	200	4	AdamW	50
YOLOv10s Grandmaster (Abanoub)	1024	200	4	AdamW	50
YOLO11n (Mario)	1024	100	4	Auto	15
YOLO11s (Mario)	1024	100	4	Auto	15
YOLO11m (Mario)	1024	100	4	Auto	15

Braille Class Mapping (Key Classes)

Classes 0–25: letters a–z. Classes 26–35: digits 1–0. Classes 36–62: capital indicator, number indicator, period, comma, question, exclamation, hyphen, apostrophe, and contractions: *and* (51), *for* (52), *of* (53), *the* (54), *with* (55), *ch* (56), *gh* (57), *sh* (58), *th* (59), *wh* (60), *ed* (61), *er* (62). (0-based YOLO indices; Angelina 1-based IDs are offset by +1.)

References

- [1] I. G. Ovodov, “Optical braille recognition using object detection neural network,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, pp. 1741–1748, 2021. Dataset available at <https://github.com/IlyaOvodov/AngelinaDataset>.
- [2] F. Agustina, E. H. Rachmawanto, N. K. D. A. Putri, F. R. Saputro, H. Lestiawan, Suprayogi, and S. Huda, “Gabor wavelet and multiclass support vector machine for braille image classification,” *Journal of Soft Computing Exploration*, vol. 5, no. 2, 2024.
- [3] S. Sana, S. Riaz, S. S. Rizvi, *et al.*, “Deep learning scheme for character prediction with position-free touch screen-based braille input method,” *Human-centric Computing and Information Sciences*, vol. 10, no. 41, 2020.
- [4] S. Shokat, S. Riaz, S. S. Rizvi, *et al.*, “Characterization of english braille patterns using automated tools and rica-based feature extraction methods,” *Sensors*, vol. 22, no. 5, p. 1836, 2022.